# Listas
# Lineares Estáticas

**Professor:** André Felipe Ribeiro Cordeiro
**Disciplina:** Estruturas de Dados

**Departamento de Informática**
**Centro de Tecnologia**
**Universidade Estadual de Maringá**

# Introdução

- Uma das formas mais simples de **interligar os elementos** de um conjunto é por meio de uma **lista**.

- Listas são **estruturas flexíveis**, que podem **crescer** ou **diminuir** de tamanho **durante a execução** de um programa.

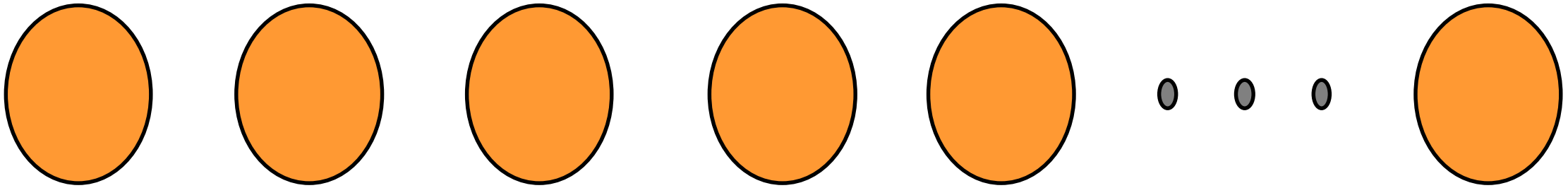
# Lista Linear

- Uma **lista linear** é uma sequência de zero ou mais itens $x_1, x_2, x_3, ..., x_n$ na qual $\mathbf{x_i}$ é de determinado tipo e **n** representa o tamanho da lista.

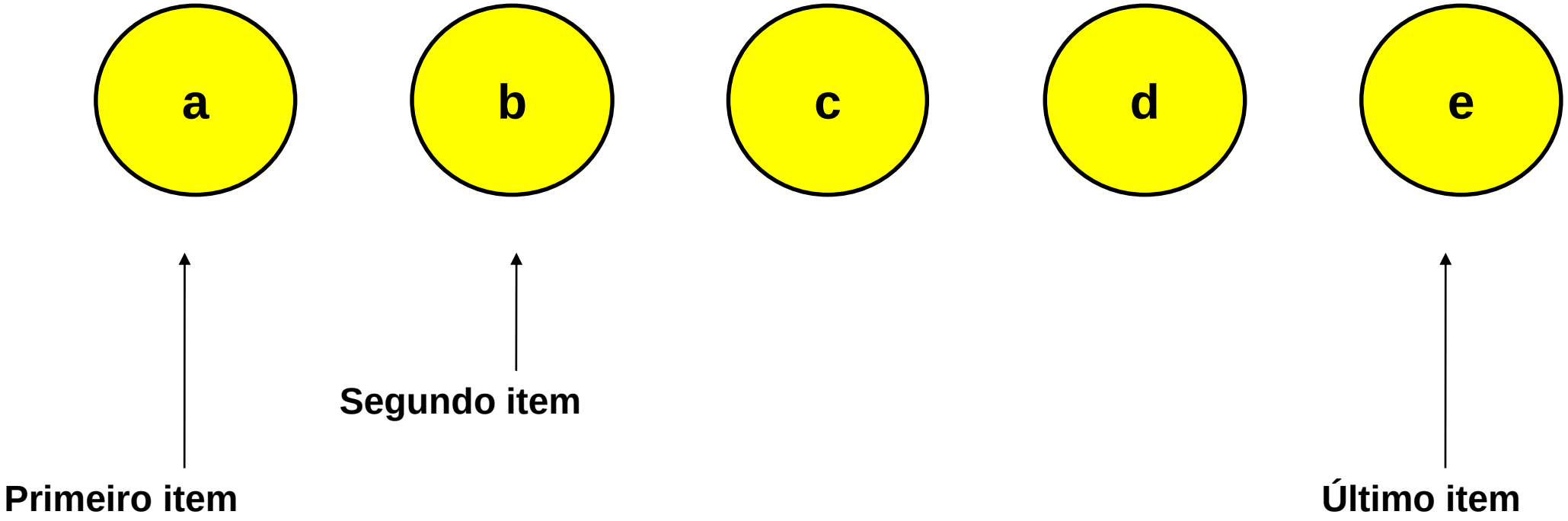
- Assumindo que $n \geq 1$, tem-se que $x_1$ é o primeiro item da lista e $x_n$ é o último item.

- Em geral, $x_i$ precede $x_{i+1}$ e sucede $x_{i-1}$.

# Exemplo



# Exemplo



# Lista Linear

- Para criar um **tipo abstrato de dados** Lista, é necessário definir um **conjunto de operações** sobre as variáveis do tipo Lista.

- O conjunto de operações a ser definido **depende de cada aplicação**, não existindo um conjunto **adequado para todas as aplicações**.

# Implementação
# Listas Lineares

- Existem **diferentes estruturas** que podem ser utilizadas na **implementação** de listas lineares.

- Cada estrutura apresenta **vantagens** e **desvantagens** particulares.

# Implementação
# Listas Lineares

- **Implementações mais utilizadas**
  - Arranjos (vetores)
  - Apontadores (ponteiros)

# Implementação
# Listas Lineares

- **Implementações mais utilizadas**
  - **Arranjos** (**vetores**)
  - Apontadores (ponteiros)

# Implementação por Arranjos

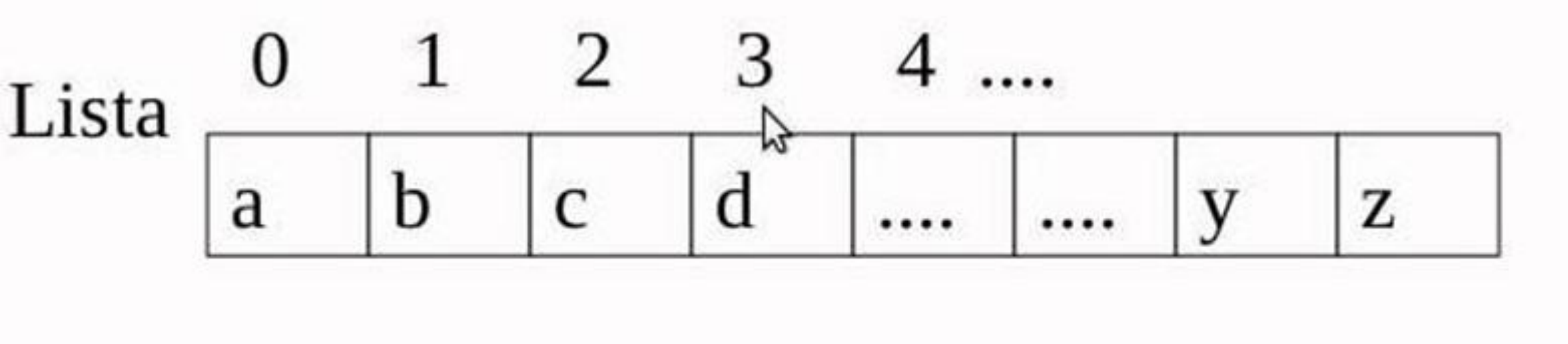
- Na **implementação por arranjos**, os itens da lista são armazenados em **posições contíguas** de memória.

- Assim, a lista pode ser **percorrida** em **qualquer direção**.

# Representação

| | Itens |
|---|---|
| Primeiro $= 1$ | $x_1$ |
| $2$ | $x_2$ |
| | $\vdots$ |
| Último$-1$ | $x_n$ |
| | $\vdots$ |
| MaxTam | |

# Representação



# Operações
# Listas Lineares

- As seguintes operações podem ser utilizadas em diferentes aplicações:
  - **inicializar** a lista
  - **verificar** se a lista está **vazia**
  - **inserir item** na lista
  - **retirar item** da lista
  - **imprimir os itens** da lista

# Implementação por Arranjos

```
#include <stdio.h>
#include <stdlib.h>

#define INICIO_ARRANJO  1
#define TAMANHO_MAXIMO  1000
```

```
typedef int TipoChave;

typedef int TipoApontador;
```

# Implementação por Arranjos

```
typedef struct {
  TipoChave chave;
  /*outros campos podem ser adicionados*/
} TipoItem;
```

```
typedef struct {
  TipoItem itens[TAMANHO_MAXIMO];
  TipoApontador primeiro, ultimo;
} TipoLista;
```

# Implementação por Arranjos

```
void Inicializa(TipoLista *lista){
    lista->primeiro = INICIO_ARRANJO;
    lista->ultimo = lista->primeiro;
}
```

```
int Vazia(TipoLista lista){
    return (lista.primeiro == lista.ultimo);
}
```

# Implementação por Arranjos

```
void Insere(TipoItem elemento,
                                TipoLista *lista){
    if (lista->ultimo > TAMANHO_MAXIMO)
        printf("Lista Cheia! \n");
    else {
        lista->itens[lista->ultimo - 1]
                                = elemento;
        lista->ultimo++;
       }
}
```

# Implementação por Arranjos

```
void Retira(TipoApontador ponteiro,
                        TipoLista *lista, TipoItem *elemento){
    int indice;

    if (Vazia(*lista) || ponteiro >= lista->ultimo){
        printf(" Erro! Posicao nao existe \n");
        return;
    }
    *elemento = lista->itens[ponteiro - 1];
    lista->ultimo--;
    for (indice = ponteiro; indice < lista->ultimo; indice++)
        lista->itens[indice - 1] = lista->itens[indice];
}
```

# Implementação por Arranjos

```
void Imprime(TipoLista lista){
    int indice;

    for (indice = lista.primeiro - 1;
                indice <= (lista.ultimo - 2); indice++)
        printf("%d\n", lista.itens[indice].chave);
}
```

# Observações

- A **inserção de um novo item** após o **último item** é realizada de maneira simples.

- Já a inserção de um novo item no **meio da lista** requer um **deslocamento** de todos os itens localizados **após** o ponto de inserção.

# Observações

- Outras operações poderiam ser consideradas, além das apresentadas.

- **Exemplos**
  - **Pesquisar** um item da lista
  - **Verificar** se a lista está cheia

# Reforço

- O **conjunto de operações** a ser definido **depende de cada aplicação**.

# Implementação por Arranjos

- **Vantagens**
  - **Transferência de dados** facilitada entre as memórias principal e secundária, uma vez que os dados estão **alocados em bloco**.
  - **Facilidade de acesso**, uma vez qualquer item da lista pode ser **diretamente acessado** a partir de um **índice** associado à sua **posição**.

# Implementação por Arranjos

- **Desvantagens**
  - Custo computacional para **inserir** ou **retirar** da lista, o que pode exigir o **deslocamento de itens**.
  - Possíveis problemas com a **alocação de espaços**, principalmente em aplicações onde **não existe previsão** sobre o **crescimento da lista**.

# Referências


- Introdução a Estruturas de Dados, Waldemar Celes, Renato Cerqueira e José Rangel, 2016.
- Projeto de Algoritmos com implementações em Pascal e C, Nivio Ziviani, 2011.
- Estruturas de Dados, Nina Edelweiss e Renata Galante, 2008.
- Aulas Professor Rodolfo Miranda Pereira, UEM.